

DSA STUDY BLUEPRINT SERIES

55. Copy List with Random Pointer

Deep Copying Lists containing Arbitrary Random pointers

Section 07 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Linked List Copying:** Clone nodes and set arbitrary pointers correctly.
- Step 1: Insert copy nodes inline. Step 2: Set random links. Step 3: Split lists.

Memory & Execution Blueprint

Original: Node A → Node B • Interleaved: Node A →
Copy A → Node B → Copy B

C++ Implementation Reference

Standard code layout and syntax

```
// Insert copy nodes inline (curr->next = new Node(curr->val, curr->next))  
// Set random links: copy->random = curr->random ? curr->random->next : nullptr
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N)$
Space Complexity	$O(1)$ auxiliary

Key Practices & Gotchas

- **Important:** Inline insertion eliminates the need to allocate key-value hash maps.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace the re-wirings of three node pointers reversing segment 10 → 20:

Step	Pointers [prev, curr, next]	Pointer reassignment	Resulting node state
1	[nullptr, 10, 20]	curr->next = prev	10 points to nullptr
2	[10, 20, nullptr]	curr->next = prev	20 points to 10 (Reversed segment completed!)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Pointer Re-linking:** Modifying node transitions requires dummy nodes to isolate head pointer alterations.
- **Fast & Slow Pointers:** Useful for loop check detections, middle-node finding, and cyclic lists splits.
- **Related LeetCode Problems:** #206 Reverse Linked List, #141 Linked List Cycle, #23 Merge k Sorted Lists.